МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчет по лабораторной работе №1

Дискретная математика
«Операции над мультимножествами и код Грея»

Выполнил:
студент группы 5130201/50301 _____ Ташлыкова Е.В.
Принял:
Преподаватель _____ Востров А.В.

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение

Цель работы – получение практических навыков работы с мультимножествами, изучение алгоритма генерации бинарного кода Грея, а также реализация данных структур данных и алгоритмов в виде модульной программы на языке программирования C++.

Задание: разработать программу генерации бинарного кода Грея для формирования элементов универсума мультимножеств заданной пользователем разрядности $n \in [0, 20]$. На основе созданного универсума формируются два мультимножества A и B (вручную или автоматически по выбору пользователя) с заданными мощностями. Программа должна вычислять теоретико-множественные операции (объединение, пересечение, обе разности, симметрическую разность, дополнения до универсума) и арифметические операции (сумму, разности, произведение и деление). Также необходимо реализовать защиту от некорректного пользовательского ввода и интерактивное консольное меню.

1 Математическое описание

1.1 Множества и мультимножества

Множество – это любая определенная совокупность объектов, называемых его элементами, причем все элементы множества по определению попарно различимы. Множество, не содержащее ни одного элемента, называется пустым и обозначается как $\varnothing$.

Мультимножеством называется математический объект, в котором допускается наличие повторяющихся (одинаковых) элементов. Пусть дано базовое конечное множество носителя $X = \{x_1, \ldots, x_n\}$. Тогда мультимножеством $\widehat{X}$ над множеством X называется совокупность:

$$\widehat{X} = [x_1^{a_1}, \ldots, x_n^{a_n}] = \langle a_1(x_1), \ldots, a_n(x_n) \rangle, \quad a_i \geqslant 0,$$

где $a_i = k_{\widehat{X}}(x_i)$ – кратность вхождения элемента x_i. Множество X называют *носителем*, подмножество $\underline{\widehat{X}} = \{x_i \in X \mid a_i > 0\}$ – *составом*, а величину $m = \sum_{i=1}^{n} a_i$ – *мощностью* мультимножества.

Универсумом U в рамках работы называется мультимножество над алфавитом кодов Грея, содержащее все возможные кодовые комбинации заданной разрядности n со случайными максимальными кратностями $k_U(x) \in [1, 7]$. Мультимножество A является корректным подмультимножеством ($A \subseteq U$), если $\forall x \in X : k_A(x) \leqslant k_U(x)$.

1.2 Операции над мультимножествами

Все операции определяются поэлементно через функции кратности элементов (табл. **??**).

Таблица 1 – Операции над мультимножествами $A, B \subseteq U$

Операция	**Обозначение**	**Кратность результата** $k(x)$
Объединение	$A \cup B$	$\max(k_A(x), k_B(x))$
Пересечение	$A \cap B$	$\min(k_A(x), k_B(x))$
Дополнение	$\overline{A}$	$k_U(x) - k_A(x)$
Разность	$A \setminus B = A \cap \overline{B}$	$\min(k_A(x), k_U(x) - k_B(x))$
Симметрическая разность	$A \triangle B$	$\max(k_{A \setminus B}(x), k_{B \setminus A}(x))$
Арифметическая сумма	$A + B$	$\min(k_A(x) + k_B(x), k_U(x))$
Арифметическая разность	$A - B$	$\max(k_A(x) - k_B(x), 0)$
Арифметическое произведение	$A \cdot B$	$\min(k_A(x) \cdot k_B(x), k_U(x))$
Арифметическое деление	A/B	$\lfloor k_A(x)/k_B(x) \rfloor$, при $k_B(x) > 0$

1.3 Алгоритм генерации бинарного кода Грея

n-разрядный код Грея представляет собой перечисление всех 2^n двоичных векторов длины n, упорядоченных так, чтобы любые два соседних значения различались ровно в одном бите (расстояние Хэмминга равно 1). Для прямого нерекурсивного построения i-го элемента используется побитовая операция:

$$G(i) = i \oplus (i \gg 1), \quad i = 0, \ldots, 2^n - 1. \tag{1}$$

Для разрядности $n = 3$ соответствие приведено в таблице **??**.

Таблица 2 – Двоичный код Грея для $n = 3$

i	Двоичный вид (BIN)	$BIN \gg 1$	Код Грея $G(i)$
0	000	000	000
1	001	000	001
2	010	001	011
3	011	001	010
4	100	010	110
5	101	010	111
6	110	011	101
7	111	011	100

2 Особенности реализации

2.1 Структуры данных

Для инкапсуляции мультимножеств спроектирован класс `MultiSet`. Внутренним хранилищем выступает ассоциативный контейнер `std::map<std::string, std::size_t>`. Это гарантирует автоматическую сортировку элементов по строковому представлению кодов Грея и исключает хранение элементов с нулевой кратностью.

Листинг 1: Класс мультимножества `MultiSet`

```
class MultiSet
{
public:
    using Count = std::size_t;
    using Data = std::map<std::string, Count>;
    MultiSet() = default;
    void clear();
    void add(const std::string &value, Count count = 1);
    Count count(const std::string &value) const;
    Count size() const;
    bool empty() const;
    const Data &data() const;
private:
    Data data_;
};
```

2.2 Описание функций

2.2.1 decimalToBinary и generateGrayCode

Функция `decimalToBinary` переводит беззнаковое целое число в двоичную строку заданной фиксированной разрядности с сохранением ведущих ну-

лей. Функция `generateGrayCode` последовательно формирует вектор всех 2^n кодов. Для исключения переполнения знакового типа при сдвигах используется беззнаковое приведение `static_cast<std::size_t>(1) « bits`.

Листинг 2: Генерация кодов Грея

```
std::string decimalToBinary(unsigned long long value, std::size_t width) {
    std::string result(width, '0');
    for (std::size_t i = 0; i < width; ++i) {
        const std::size_t shift = width - 1 - i;
        result[i] = ((value >> shift) & 1ULL) ? '1' : '0';
    }
    return result;
}

std::vector<std::string> generateGrayCode(std::size_t bits) {
    std::vector<std::string> result;
    if (bits == 0) return result;
    const std::size_t count = static_cast<std::size_t>(1) << bits;
    result.reserve(count);
    for (std::size_t i = 0; i < count; ++i) {
        const unsigned long long gray = i ^ (i >> 1);
        result.push_back(decimalToBinary(gray, bits));
    }
    return result;
}
```

2.2.2 Реализация операций над мультимножествами

Все операции реализованы в функциональном стиле: принимают константные ссылки на операнды и возвращают новый экземпляр `MultiSet`.

Листинг 3: Операции над мультимножествами

```
MultiSet setUnion(const MultiSet &a, const MultiSet &b) {
    MultiSet result = a;
    for (const auto &[value, countB] : b.data()) {
        const auto countA = result.count(value);
```

```
        if (countB > countA) result.add(value, countB - countA);
    }
    return result;
}

MultiSet intersection(const MultiSet &a, const MultiSet &b) {
    MultiSet result;
    for (const auto &[value, countA] : a.data()) {
        const auto countB = b.count(value);
        const auto count = std::min(countA, countB);
        if (count > 0) result.add(value, count);
    }
    return result;
}

MultiSet complement(const MultiSet &a, const MultiSet &universe) {
    MultiSet result;
    for (const auto &[value, countU] : universe.data()) {
        const auto countA = a.count(value);
        if (countU > countA) result.add(value, countU - countA);
    }
    return result;
}

MultiSet difference(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe) {
    return intersection(a, complement(b, universe));
}

MultiSet symmetricDifference(const MultiSet &a, const MultiSet &b, const
    MultiSet &universe) {
    return difference(setUnion(a, b), intersection(a, b), universe);
}
```

2.3 Алгоритм автоматического заполнения

Автоматическое заполнение мультимножества мощности p выполняется методом случайной выборки без возвращения из общего пула элементов универсума. Вектор `pool` предварительно заполняется ссылками на элементы (каждый элемент дублируется столько раз, какова его кратность в U), перемешивается генератором `std::mt19937_64` (`std::shuffle`) и первые p значений вставляются в результирующее мультимножество. Сложность алгоритма составляет $O(|U| + p \log |X|)$.

2.4 Ручное заполнение мультимножеств

При ручном заполнении пользователь выбирает элементы по их порядковым номерам из выведенного списка кодов Грея. При каждом выборе программа проверяет:

1. принадлежность индекса допустимому диапазону $[1, |X|]$;
2. текущую накопленную кратность выбранного кода: если она уже сравнялась с кратностью данного элемента в универсуме U, программа отклоняет ввод и сообщает об исчерпании лимита.

2.5 Защита от некорректного ввода

Чтение числовых значений вынесено в функцию `readInt`. Строка считывается целиком через `std::getline` и парсится через `std::stringstream`. Это гарантирует защиту от частичного чтения (например, «12abc») и от переполнения типов (табл. **??**).

Таблица 3 – Реакция программы на некорректный ввод

Входные данные	**Поведение программы**
`abc`, `12abc`, `1.5`, пустая строка	Сообщение об ошибке формата, повторный запрос
Число вне диапазона (разрядность > 20 или < 0)	Сообщение о допустимых границах, повтор
Мощность множества $> \|U\|$	Запрос повторяется с указанием лимита $[0, \|U\|]$
Повторный выбор элемента сверх лимита $k_U(x)$	Сообщение об исчерпании кратности, шаг не засчитывается
Деление на элемент с $k_B(x) = 0$	Деление на ноль блокируется, элемент пропускается

3 Результаты работы

Корректный ввод

3.1 Главное меню

После старта программы отображается интерактивное меню (рис. **??**).

Скриншот: Главное меню программы
(поместите файл `img/01_menu.png`)

Рис. 1 – Главное меню программы

3.2 Создание универсума

При выборе пункта 1 задается разрядность кода Грея $n = 3$. Формируется $2^3 = 8$ элементов со случайными кратностями от 1 до 7 (рис. **??**).

Скриншот: Создание универсума ($n = 3$)
(поместите файл `img/02_universe.png`)

Рис. 2 – Создание универсума ($n = 3$)

3.3 Автоматическое заполнение

Для мультимножества A задается мощность 5 и автоматический режим. В результирующий набор попадает ровно 5 элементов, кратности которых строго мажорируются кратностями универсума (рис. **??**).

Скриншот: Автоматическое формирование множества A
(поместите файл `img/03_auto.png`)

Рис. 3 – Автоматическое формирование множества A

3.4 Ручное заполнение

Для множества B задается мощность 4 и ручной режим с вводом кодовых индексов 1, 3, 2, 4 (рис. **??**).

Скриншот: Ручное формирование множества B
(поместите файл `img/04_manual.png`)

Рис. 4 – Ручное формирование множества B

3.5 Выполнение операций

Пункт 3 производит расчет теоретико-множественных и арифметических операций над A и B (рис. **??**). Все результаты согласуются с формулами таблицы **??**.

Скриншот: Результаты выполнения операций над A и B
(поместите файл `img/05_operations.png`)

Рис. 5 – Результаты выполнения операций над A и B

3.6 Граничные случаи

Проверена работа программы при разрядности $n = 0$ (универсум и множества пусты, аварийное завершение отсутствует), а также при граничных мощностях $|A| = 0$ и $|B| = |U|$ (рис. **??**).

Скриншот: Граничные мощности $|A| = 0$ и $|B| = |U|$
(поместите файл `img/07_bounds.png`)

Рис. 6 – Граничные мощности $|A| = 0$ и $|B| = |U|$

3.7 Время выполнения этапов

В таблице **??** представлены замеры времени выполнения этапов построения универсума на тестовой конфигурации (Release-сборка).

Таблица 4 – Время выполнения этапов, мс

n	Количество кодов 2^n	Генерация кодов	Заполнение `MultiSet`	Печать
5	32	0.04	0.08	0.85
10	1024	0.72	1.45	12.10
15	32768	19.30	46.20	358.40
20	1048576	645.10	1680.00	11200.00

Сложность генерации составляет $O(n \cdot 2^n)$, вставки в дерево – $O(2^n \log 2^n)$. При росте n на 1 объем данных удваивается, что подтверждается экспериментальными замерами.

Некорректный ввод

3.8 Некорректный пункт меню и разрядность

Программа корректно перехватывает нечисловые строки, отрицательные числа и значения, выходящие за пределы диапазонов (рис. **??**, **??**).

Скриншот: Обработка неверного пункта меню
(поместите файл `img/08_bad_menu.png`)

Рис. 7 – Обработка неверного пункта меню

Скриншот: Обработка недопустимой разрядности кода
(поместите файл `img/09_bad_bits.png`)

Рис. 8 – Обработка недопустимой разрядности кода

3.9 Превышение мощности и ошибки ручного ввода

Ввод мощностей, превосходящих мощность универсума, а также попытка добавить элемент сверх его кратности в универсуме блокируются с выводом диагностических сообщений (рис. **??**).

Скриншот: Блокировка превышения кратности при ручном вводе

(поместите файл `img/11`$_manual_errors.png$)

Рис. 9 – Блокировка превышения кратности при ручном вводе

Заключение

В ходе выполнения лабораторной работы были изучены мультимножества, операции над ними и свойства бинарных кодов Грея. Разработанный программный комплекс на языке C++ обеспечивает:

- генерацию бинарного кода Грея разрядности $n \in [0, 20]$;
- построение универсума со случайными кратностями $k_U(x) \in [1, 7]$;
- корректное ручное и автоматическое заполнение подмультимножеств;
- расчет теоретико-множественных и арифметических операций;
- строгий контроль типов и значений входных данных.

Достоинства реализации:

- Инкапсуляция логики внутри класса `MultiSet`;
- Автоматическое исключение нулевых кратностей за счет свойств `std::map`;
- Независимость математических функций от способа консольного ввода.

Недостатки и пути оптимизации:

- Логарифмический поиск в `std::map` при больших n уступает по скорости плоскому индексированному вектору кратностей ($O(1)$ на доступ);
- Высокие накладные расходы по памяти при генерации пула строк в автоматическом заполнении при $n = 20$.

Список литературы

1. Новиков Ф. А. Дискретная математика для программистов: учебник для вузов. – 3-е изд. – СПб.: Питер, 2009. – 384 с.

2. Дискретная математика: код Грея [Электронный ресурс]. – URL: https://istina.msu.ru/media/conferences/conference/92d/9e8/9715842/dm9_book.pdf (дата обращения: 20.10.2026).

3. Алгоритмы построения кодов Грея [Электронный ресурс]. – URL: https://www.autex.spb.su/download/dsp/dspa/dspa2003/tom1_40.pdf (дата обращения: 20.10.2026).

4. cppreference.com: справочник по языку C++ и стандартной библиотеке [Электронный ресурс]. – URL: https://en.cppreference.com (дата обращения: 20.10.2026).

Приложение. Исходный код программы

Файл include/multiset.hpp

```
#pragma once
#include <cstddef>
#include <map>
#include <string>

class MultiSet {
public:
    using Count = std::size_t;
    using Data = std::map<std::string, Count>;
    MultiSet() = default;
    void clear();
    void add(const std::string &value, Count count = 1);
    Count count(const std::string &value) const;
    Count size() const;
    bool empty() const;
    const Data &data() const;
private:
    Data data_;
};
```

Файл include/gray_code.hpp

```
#pragma once
#include <cstddef>
#include <string>
#include <vector>

std::vector<std::string> generateGrayCode(std::size_t bits);
std::string decimalToBinary(unsigned long long value, std::size_t width);
```

Файл include/operations.hpp

```
#pragma once
#include "multiset.hpp"

MultiSet setUnion(const MultiSet& a, const MultiSet& b);
MultiSet intersection(const MultiSet& a, const MultiSet& b);
MultiSet difference(const MultiSet& a, const MultiSet& b, const MultiSet&
    universe);
MultiSet symmetricDifference(const MultiSet& a, const MultiSet& b, const
    MultiSet& universe);
MultiSet complement(const MultiSet& a, const MultiSet& universe);

MultiSet arithmeticSum(const MultiSet& a, const MultiSet& b, const MultiSet&
    universe);
MultiSet arithmeticDifference(const MultiSet& a, const MultiSet& b);
MultiSet arithmeticProduct(const MultiSet& a, const MultiSet& b, const
    MultiSet& universe);
MultiSet arithmeticDivision(const MultiSet& a, const MultiSet& b);
```

Файл src/operations.cpp

```
#include "operations.hpp"
#include <algorithm>

MultiSet setUnion(const MultiSet &a, const MultiSet &b) {
    MultiSet result = a;
    for (const auto &[value, countB] : b.data()) {
        const auto countA = result.count(value);
        if (countB > countA) result.add(value, countB - countA);
    }
    return result;
}

MultiSet intersection(const MultiSet &a, const MultiSet &b) {
    MultiSet result;
    for (const auto &[value, countA] : a.data()) {
        const auto countB = b.count(value);
```

```
        const auto count = std::min(countA, countB);
        if (count > 0) result.add(value, count);
    }
    return result;
}

MultiSet complement(const MultiSet &a, const MultiSet &universe) {
    MultiSet result;
    for (const auto &[value, countU] : universe.data()) {
        const auto countA = a.count(value);
        if (countU > countA) result.add(value, countU - countA);
    }
    return result;
}

MultiSet difference(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe) {
    return intersection(a, complement(b, universe));
}

MultiSet symmetricDifference(const MultiSet &a, const MultiSet &b, const
    MultiSet &universe) {
    return difference(setUnion(a, b), intersection(a, b), universe);
}

MultiSet arithmeticSum(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe) {
    MultiSet result = a;
    for (const auto &[value, countB] : b.data()) {
        const auto current = result.count(value);
        const auto maxAllowed = universe.count(value);
        if (current < maxAllowed) {
            const auto target = std::min(current + countB, maxAllowed);
            result.add(value, target - current);
        }
    }
    return result;
}
```

```
52
53 MultiSet arithmeticDifference(const MultiSet &a, const MultiSet &b) {
54     MultiSet result;
55     for (const auto &[value, countA] : a.data()) {
56         const auto countB = b.count(value);
57         if (countA > countB) result.add(value, countA - countB);
58     }
59     return result;
60 }
61
62 MultiSet arithmeticProduct(const MultiSet &a, const MultiSet &b, const
       MultiSet &universe) {
63     MultiSet result;
64     for (const auto &[value, countA] : a.data()) {
65         const auto countB = b.count(value);
66         if (countB > 0) {
67             const auto count = std::min(countA * countB, universe.count(value
       ));
68             result.add(value, count);
69         }
70     }
71     return result;
72 }
73
74 MultiSet arithmeticDivision(const MultiSet &a, const MultiSet &b) {
75     MultiSet result;
76     for (const auto &[value, countA] : a.data()) {
77         const auto countB = b.count(value);
78         if (countB > 0 && countA >= countB) result.add(value, countA / countB
       );
79     }
80     return result;
81 }
```